

Project #9: The Rail Fence Cipher

Confederate officers and spies were pretty much on their own when it came to cryptography. This led to unsophisticated solutions, one of the favorites being the rail fence cipher, so named due to its resemblance to the zigzag pattern of a split-rail fence (shown in Figure 4-8).



Figure 4-8: A rail fence

The rail fence is a simple-to-use transposition cipher, like the Union's route cipher, but differs from the route cipher in that it transposes letters rather than words, making it more error-prone. And since the number of possible keys is much more restrictive than the number of paths through a route cipher, the rail fence cipher is much easier to "tear down."

Both the Union and Confederates used the rail fence as a field cipher, and the spies probably didn't use code words very often. Codebooks needed to be tightly controlled, for obvious reasons, and were more likely to be secured in a military telegraph office than carried around by easily compromised undercover agents.

Sometimes the Confederates used the more complex Vigenère cipher (see "Project #12: Hiding a Vigenère Cipher" on page 106) for important messages—and some unimportant ones to mislead enemies—but it was tedious work to decipher and equally laborious to encrypt and not suitable for fast field communications.

Despite their lack of training in the mechanics of cryptography, the Confederacy, and Southerners in general, were clever and innovative. Among their more impressive accomplishments in the art of secret messages was the use of microphotography, 100 years before it was widely adopted during the Cold War.

THE OBJECTIVE

Write Python programs that will help a spy encrypt and decrypt a “two-rail” (two-row) rail fence cipher. You should write the programs in a way that will reduce potential errors by inexperienced users.

The Strategy

To encrypt a message with the rail fence cipher, follow the steps in Figure 4-9.

Buy more Maine potatoes	1) Write plaintext
BUYMOREMAINEPOTATOES	2) Remove spaces and capitalize
 B Y O E A N P T T E U M R M I E O A O S	3) Stack and stagger letters in zigzag pattern
BYOEANPTTEUMRMIEOAOS	4) Merge the upper and lower rows
BYOEA NPTTE UMRMI EOAOS	5) Split into groups of five

Figure 4-9: Encryption process for a “two-rail” rail fence cipher

After the plaintext is written, the spaces are removed, and all the letters are converted to uppercase (Step 2). Using uppercase is common convention in cryptography, as it obfuscates the presence of proper names and the beginning of sentences, giving a cryptanalyst fewer clues for deciphering the message.

The message is then written in stacked fashion, with every other letter below the previous letter and shifted over one space (Step 3). This is where the “rail fence” analogy becomes apparent.

The first row is then written, followed immediately by the second row on the same line (Step 4), and then the letters are broken into groups of five to create the illusion of distinct words and to further confuse the cryptanalyst (Step 5).

To decrypt a rail fence cipher, reverse the process. Just remove the spaces, divide the message in half, stack the second half below the first, offset by one letter, and read the message using the zigzag pattern. If the ciphertext has an odd number of letters, put the extra letter in the first (upper) half.

To make things easy for people who want to use a rail fence cipher, follow the preceding steps to write two programs, one to encrypt and another to decrypt. Figure 4-9 is essentially your pseudocode, so let’s get to it. And since you now know how to package the code for inexperienced users, take that approach from the start.

The Rail Fence Cipher Encryption Code

The code in this section allows a user to enter a plaintext message and have the encrypted results print in the interpreter window. This code is available for download with the book's resources at <https://www.nostarch.com/impracticalpython/>.

Instructing the User and Getting Input

Listing 4-6, at the top of *rail_fence_cipher_encrypt.py*, provides the program's instructions and assigns the plaintext to a variable.

```
rail_fence_cipher_encrypt.py,
part 1

❶ r"""Encrypt a Civil War 'rail fence' type cipher.

This is for a "2-rail" fence cipher for short messages.

Example text to encrypt: 'Buy more Maine potatoes'

Rail fence style: B Y O E A N P T T E
                  U M R M I E O A O S

Read zigzag:      \/\/\/\/\/\/\/\/\/

Encrypted:  BYOEA NPTTE UMRMI EOSOS

"""
#-----
❷ # USER INPUT:

# the string to be encrypted (paste between quotes):
❸ plaintext = """Let us cross over the river and rest under the shade of the trees
"""

❹ # END OF USER INPUT - DO NOT EDIT BELOW THIS LINE!
#-----
```

Listing 4-6: Docstring and user input section for *rail_fence_cipher_encrypt.py*

Start with a multiline docstring, placing an `r` (which stands for “raw”) prefix before the first set of triple quotes ❶. Without this prefix, Pylint will complain bitterly about the `\\` slashes used further down. Fortunately, `pydocstyle` points this out so you can fix it (read Chapter 1 to learn all about Pylint and `pydocstyle`). If you want to know more about *raw strings*, see Section 2.4.1 in the Python docs (https://docs.python.org/3.6/reference/lexical_analysis.html#string-and-bytes-literals).

Next, separate the program's docstring and `import` statements from the input section with a line and let the user know they're on deck with an all-caps comment ❷. Clearly define the input requirement with comments and place the plaintext inside triple quotes to better accommodate long text strings ❸.

Finally, end the user input section with a declaration to that effect and caution to not edit anything below the following line ❹.

Encrypting a Message

Add Listing 4-7 to *rail_fence_cipher_encrypt.py* in order to handle the encryption processes.

```
rail_fence_cipher_encrypt.py, part 2
❶ def main():
    """Run program to encrypt message using 2-rail rail fence cipher."""
    message = prep_plaintext(plaintext)
    rails = build_rails(message)
    encrypt(rails)

❷ def prep_plaintext(plaintext):
    """Remove spaces & leading/trailing whitespace."""
    ❸ message = "".join(plaintext.split())
    ❹ message = message.upper() # convention for ciphertext is uppercase
    print("\nplaintext = {}".format(plaintext))
    return message

❺ def build_rails(message):
    """Build strings with every other letter in a message."""
    evens = message[::2]
    odds = message[1::2]
    ❻ rails = evens + odds
    return rails

❼ def encrypt(rails):
    """Split letters in ciphertext into chunks of 5 & join to make string."""
    ❽ ciphertext = ' '.join([rails[i:i+5] for i in range(0, len(rails), 5)])
    print("ciphertext = {}".format(ciphertext))

❾ if __name__ == '__main__':
    main()
```

Listing 4-7: Defines functions to encrypt the plaintext message

To start, define a `main()` function to run the program ❶. Having a `main()` function gives you the flexibility to use this program as a module in another program later, should the need arise. This function calls three other functions: one to prepare the input plaintext, one to build the “rails” used by the cipher, and one to break the encrypted text into five-letter chunks.

Next, define a function to take the input string and prepare it for encryption ❷. This involves removing spaces ❸ and converting letters to uppercase (as in Step 2 in Figure 4-9) ❹. Then, after a newline, print the plaintext to the screen and return it.

Now, define a function to build two strings, as in Step 3 of Figure 4-9, by slicing `message` for evens (starting at 0 and stepping by 2) and for odds (starting at 1 and stepping by 2) ❺. The two strings are then concatenated in a new string, named `rails` ❻, and returned.

Define an encryption function that takes the `rails` string as an argument ❼. Use list comprehension to split the ciphertext into chunks of five (as in Step 5 of Figure 4-9) ❽. The encrypted text is then printed to the screen. Finish with the code to run the program as a module or in stand-alone mode ❾.

Here is the output from this program:

```
plaintext = Let us cross over the river and rest under the shade of the trees
ciphertext = LTSRS OETEI EADET NETEH DOTER EEUCO SVRHR VRNRS UDRHS AEFHT ES
```

The Rail Fence Cipher Decryption Code

The code in this section allows a user to enter a message encrypted with the rail fence cipher and have the plaintext printed in the interpreter window. This code is available for download along with the rest of the book's resources at <https://www.nostarch.com/impracticalpython/>.

Importing Modules, Instructing the User, and Getting Input

Listing 4-8 starts with instructions similar to those in the *rail_fence_cipher_encrypt.py* program (Listing 4-6), imports two modules, and gets the user input.

rail_fence_cipher_decrypt.py,
part 1

```
"""Decrypt a Civil War 'rail fence' type cipher.

This is for a 2-rail fence cipher for short messages.

Example plaintext: 'Buy more Maine potatoes'

Rail fence style:  B Y O E A N P T T E
                   U M R M I E O A O S

Read zigzag:      \/\/\/\/\/\/\/\/\/

Ciphertext:  BYOEA NPTTE UMRMI EOSOS

"""
❶ import math
import itertools

#-----
# USER INPUT:

# the string to be decrypted (paste between quotes):
❷ ciphertext = """LTSRS OETEI EADET NETEH DOTER EEUCO SVRHR VRNRS UDRHS AEFHT ES
"""

# END OF USER INPUT - DO NOT EDIT BELOW THIS LINE!
#-----
```

Listing 4-8: Imports modules, instructs the user, and gets user input

One difference here is that you need to import the `math` and `itertools` modules ❶. You'll use `math` for rounding. The `itertools` module is a group of

functions in the Python Standard Library that create iterators for efficient looping. You'll use `itertools.zip_longest()` function during the decryption process.

The only other change is that the user should enter ciphertext, rather than plaintext ❷.

Decrypting a Message

Listing 4-9 defines the functions for preparing and decoding the ciphertext and finishes off *rail_fence_cipher_decrypt.py*.

```
rail_fence_cipher ❶ def main():
_decrypt.py,      """Run program to decrypt 2-rail rail fence cipher."""
part 2            message = prep_ciphertext(ciphertext)
                  row1, row2 = split_rails(message)
                  decrypt(row1, row2)

❷ def prep_ciphertext(ciphertext):
    """Remove whitespace."""
    message = "".join(ciphertext.split())
    print("\nciphertext = {}".format(ciphertext))
    return message

❸ def split_rails(message):
    """Split message in two, always rounding UP for 1st row."""
    ❹ row_1_len = math.ceil(len(message)/2)
    ❺ row1 = (message[:row_1_len]).lower()
    row2 = (message[row_1_len:]).lower()
    return row1, row2

❻ def decrypt(row1, row2):
    """Build list with every other letter in 2 strings & print."""
    ❼ plaintext = []
    ❽ for r1, r2 in itertools.zip_longest(row1, row2):
        plaintext.append(r1)
        plaintext.append(r2)
    ❾ if None in plaintext:
        plaintext.pop()
    print("rail 1 = {}".format(row1))
    print("rail 2 = {}".format(row2))
    print("\nplaintext = {}".format(''.join(plaintext)))

❿ if __name__ == '__main__':
    main()
```

Listing 4-9: Prepares, decodes, and prints the message

The `main()` function here ❶ is similar to the one used in the encryption program in Listing 4-7. Three functions are called: one to prepare the input string, one to “split the rails” in the rail fence cipher, and one to stitch the two rails back together into readable plaintext.

Start with a function that repeats the preprocessing steps used during encryption ❷. Remove the spaces between the five-letter chunks, as well as any other whitespace created during the pasting of the ciphertext, and print and return the ciphertext.

Next, you need to split the message back into two halves to reverse the encryption process ❸. As I mentioned in “The Strategy” on page 81, the extra letter in a message with an odd number of characters is assigned to the top row. To address the odd-numbered case, use the `math.ceil()` method ❹. “Ceil” stands for “ceiling,” so when you divide by 2, the answer is always rounded up to the nearest integer. Assign this number to the `row_1_len` variable. Once you know the length of the first row, use that value and slicing to divide the `message` variable into two strings representing the rows ❺. End the function by returning the row variables.

Now it’s just a matter of choosing and joining every other letter from the rows to stitch the plaintext back together. Define a `decrypt()` function and pass it the strings for `row1` and `row2` ❻. Start the translation process by making an empty list to hold the results ❼. Next, you need an easy way to deal with cases where the ciphertext has an odd number of letters—resulting in two rows of different lengths—because Python prevents you from looping through two uneven sequences by raising an index-out-of-range error. This is why we’ve imported the `itertools` module—its functions help with looping to circumvent this problem.

The `itertools.zip_longest()` function accepts two strings as arguments and loops through them without complaint, appending a null value (`None`) to the plaintext list when it gets to the end of the shorter string ❽. You don’t want to print this null value, so if it’s there, remove it using the `pop()` method you applied in the route cipher code ❾. Complete the decryption process by printing the two rows (rails) to the screen, followed by the decrypted ciphertext.

End with the standard code for running the program as a module or in stand-alone mode ❿. The output from the program is as follows:

```
ciphertext = LTSRS OETEI EADET NETEH DOTER EEUCO SVRHR VRNRS UDRHS AEFHT ES

rail 1 = LTSRSOETEI EADETNETEHDOTERE
rail 2 = EUCOSVRHRVRNRSUDRHS AEFHTES

plaintext = letuscrossovertheriverandrestundertheshadeofthetrees
```

Note that there will be no spaces between words, but that’s okay—you don’t want to leave the cryptanalyst feeling completely useless!

Summary

That completes our foray into Civil War ciphers. You wrote a program that helps a user decrypt a route cipher, and you gained valuable insights into how it works and how to hack it. You can implement an automated attack on

the cipher in the following practice projects, but remember, with its many possible paths and use of code words, the Union route cipher is a tough nut to fully crack.

Next, you wrote programs to encrypt and decrypt two-rail fence ciphers. Given how tedious and error-prone the manual encryption and decryption processes are, having an automated way to do most of the work would have been valuable to both sides in the war. And to further address these types of issues, you wrote your code to be user-friendly to the inexperienced cryptanalyst or spy.

Further Reading

More beginner-level Python programs for working with transposition ciphers are available in *Cracking Codes with Python* (No Starch Press, 2018) by Al Sweigart.

Excellent and well-illustrated overviews of cryptography can be found in *Mysterious Messages: A History of Codes and Ciphers* (The Penguin Group, 2009) by Gary Blackwood and *The Code Book: The Science of Secrecy from Ancient Egypt to Quantum Cryptography* (Anchor, 2000) by Simon Singh.

The sites <http://www.civilwarsignals.org/pages/crypto/crypto.html> and http://www.mathaware.org/mam/06/Sauerberg_route-essay.html include descriptions of Edward Porter Alexander's attempt to solve a route cipher. Alexander was the father of the Confederate Army Signal Corps and a brilliant military innovator with many impressive accomplishments.

Practice Projects

Hone your cryptography skills with these projects. Solutions are available in the appendix and online.

Hacking Lincoln

In his book, *Mysterious Messages: A History of Codes and Ciphers*, Gary Blackwood reproduces an actual message sent by Abraham Lincoln and encrypted with a route cipher:

**THIS OFF DETAINED ASCERTAIN WAYLAND CORRESPONDENTS
OF AT WHY AND IF FILLS IT YOU GET THEY NEPTUNE THE
TRIBUNE PLEASE ARE THEM CAN UP**

Use the `route_cipher_decrypt.py` program to solve this cryptogram. The number of columns and rows must be factors of the message length, and the route starts in one of the corners, doesn't skip columns, and alternates direction with every column change. The code word definitions and plaintext solution can be found in the appendix.